

Day 2 Summary — Security Foundation & JWT Implementation

LOS Wallet Settlement Ecosystem

■ Objective

Membangun pondasi keamanan (Security Foundation) untuk ekosistem LOS Wallet Settlement yang scalable, modular, dan sesuai best practice enterprise. Fokus hari ini mencakup setup Spring Security modern (Spring Boot 3 / Spring Security 6), implementasi JWT-based Authentication, dan menyiapkan struktur dasar untuk Authorization.

■ Architecture Overview

Security layer ini menjadi bagian dari service-auth module dan berperan sebagai Authentication Provider, Token Issuer, dan Security Filter Layer.

■ Spring Security — Core Concepts

1■■ Authentication

Proses memverifikasi identitas user. Spring Security menggunakan AuthenticationManager dan UserDetailsService untuk melakukan autentikasi.

2■■ Authorization

Menentukan apa yang boleh dilakukan oleh user yang sudah login. Dapat diatur melalui method-level security atau konfigurasi HttpSecurity DSL.

3■■ PasswordEncoder

Digunakan untuk hashing password. Biasanya memakai BCryptPasswordEncoder untuk keamanan optimal.

■ JWT Overview

JWT adalah stateless authentication mechanism yang digunakan agar server tidak perlu menyimpan session di database. Struktur JWT terdiri dari HEADER.PAYLOAD.SIGNATURE dengan algoritma HS256 atau RS256.

■ Best Practice JWT Implementation

- Gunakan algoritma HS512 / RS256
- Access Token lifetime 15–30 menit
- Simpan hanya informasi penting (username, roles)
- Hindari commit secret key di Git
- Jangan simpan session agar tetap stateless

■■ Sample Token Provider

```
@Component @RequiredArgsConstructor public class TokenProvider { @Value("${app.jwt.secret}")
private String secret; @Value("${app.jwt.expiration}") private long expiration; public String
createAccessToken(String username, List roles, Map claims) { Date now = new Date(); Date expiry = new
Date(now.getTime() + expiration); return Jwts.builder() .setSubject(username) .addClaims(claims != null ?
claims : new HashMap<>()) .claim("roles", roles) .setIssuedAt(now) .setExpiration(expiry)
.signWith(Keys.hmacShaKeyFor(secret.getBytes()), SignatureAlgorithm.HS512) .compact(); } public String
getUsername(String token) { return Jwts.parserBuilder() .setSigningKey(secret.getBytes()) .build()
.parseClaimsJws(token) .getBody() .getSubject(); } }
```

■ Key Insight

Spring Security modern encourages declarative configuration, short-lived tokens, and clear separation between authentication (who you are) and authorization (what you can do).

■ Next Step (Day 3 Preview)

- Integrate Flyway untuk database schema versioning
- Setup User + Role table migration
- Tambah Refresh Token logic
- Implementasi GlobalExceptionHandler untuk response error yang clean